KOGI

Independent Worker Operating System

Request for Comments

RFC-001 through RFC-008 · Engineering & Product Proposals

Collection v1.0 · Platform Architecture

Document Type	Request for Comments (RFC) Collection
Version	1.0 — Initial Collection
Status	Mixed — see individual RFC status headers
RFC Count	8 RFCs covering core architectural and product decisions
Companion Docs	PRD v1.0 · PRS v1.0 · LLSDS v0.1
Review Process	RFC author distributes to stakeholders → 2-week comment period → Decision meeting → Status updated
Decision Authority	Technical RFCs: CTO. Product RFCs: CPO. Cross-functional: both with CEO tiebreak

ℹ️ **RFC Process**

- An RFC (Request for Comments) is a proposal document submitted when a significant technical or product decision must be made that affects multiple teams or long-term platform architecture.
- An RFC is NOT required for: bug fixes, minor feature additions within an existing module, or implementation details within a single team's scope.
- An RFC IS required for: new modules, cross-module data model changes, new external dependencies, security architecture changes, pricing model changes, and anything that cannot easily be reversed.

- Any engineer, PM, or designer may author an RFC. RFC numbers are assigned sequentially.

RFC Index

RFC	Title	Status	Author	Category
RFC-001	Event-Driven Architecture with Outbox Pattern	Accepted	Platform Arch Team	Architecture
RFC-002	Financial Transaction Safety Model — Ledger & Escrow Design	Accepted	Exchange Team	Architecture
RFC-003	AI Agent Context & Memory Architecture	Draft	AI Platform Team	Architecture
RFC-004	Marketplace Seeding & Liquidity Bootstrap Strategy	Accepted	Product + Market Team	Product
RFC-005	Cooperative & AO Governance Engine Design	Draft	AO Team	Architecture
RFC-006	Monorepo, Build System, and Service Extraction Strategy	Accepted	Platform Arch Team	Engineering
RFC-007	Freemium Tier Definition & Feature Gating Architecture	Accepted	Product + Engineering	Product
RFC-008	Real-Time Collaboration Architecture (WebSocket vs. SSE)	Accepted	Community Team	Architecture

RFC-001 · ARCHITECTURE

Event-Driven Architecture with Outbox Pattern

RFC-001

Event-Driven Architecture with Outbox Pattern

Status: Accepted **Author:** Platform Architecture Team **Created:** 2024-06-01 **Updated:** 2024-07-15

Summary

This RFC proposes adopting an event-driven architecture using the Transactional Outbox pattern as the primary mechanism for cross-module communication across all Kogi services. This decision affects the entire platform and is foundational for all future module development.

1. Motivation

Kogi's 12 modules must communicate to maintain data consistency. For example, when a market order is placed, the Exchange module must create an escrow, the Portfolio module must create a linked item, and the WBS module must create a linked story — all as a result of a single user action.

Synchronous, point-to-point RPC between modules (e.g. market-svc calls exchange-svc calls portfolio-svc in a chain) creates tight coupling, cascading failures, and makes the system brittle. If the portfolio-svc is down, the order cannot be placed even though the order and escrow logic is unaffected.

2. Proposal

All cross-module communication shall be asynchronous and event-driven. The Transactional Outbox pattern guarantees that events are always published even if the publishing service crashes between the database commit and the message broker publish.

2.1 Transactional Outbox Pattern

The outbox pattern works as follows: when a service writes a domain state change to its database, it simultaneously writes an event record to an outbox_events table in the same transaction. A separate relay process then reads pending outbox events and publishes them to the message broker (NATS JetStream). This guarantees at-least-once delivery — the event will be published eventually, even if the service crashes immediately after the DB commit.

ℹ️ **Outbox Pattern — Key Properties**

- Atomicity: domain write + outbox event insert are in the same DB transaction — they either both succeed or both fail

- Guaranteed delivery: relay process retries until NATS acknowledges receipt — no events are silently dropped
- At-least-once: consumers must be idempotent (handle duplicate events gracefully)
- Ordering: events for a single entity (e.g. portfolio_id=XYZ) are ordered within that entity's stream
- Backpressure: NATS consumer lag is monitored; slow consumers do not cause head-of-line blocking

3. Alternatives Considered

Option	Description	Pros	Cons	Verdict
A — Outbox(Proposed)	DB transaction + outbox table + relay process → NATS	Guaranteed delivery; decoupled services; survives partial failures; simple consumer recovery	Additional infrastructure (relay process per service); slight latency vs. synchronous RPC	✅ Accepted
B — Direct NATS publish	Service publishes to NATS directly after DB commit, without outbox table	Simple; no relay process; low latency	Dual-write problem: if service crashes after DB commit but before NATS publish, event is lost permanently	❌ Rejected — financial data requires guaranteed delivery
C — Synchronous RPC	market-svc calls exchange-svc, portfolio-svc, wbs-svc via gRPC in sequence	Simple to reason about; strong consistency	Cascading failures; tight coupling; unacceptable availability impact when any dependency is down	❌ Rejected — violates availability requirements
D — CDC (Debezium)	Capture DB changes via Postgres replication log → Kafka	No application-level outbox code; low overhead	Complex infra (Kafka + Debezium); schema coupling between DB and event schema; harder to manage event schema evolution	⚠️ Deferred — may adopt for analytics/OLAP at scale; not for primary event bus

4. Implementation Notes

- Every service DB requires: outbox_events(id, event_type, payload JSONB, status, created_at, published_at, retry_count)
- Relay process runs per service with a 100ms poll interval. Uses SELECT ... FOR UPDATE SKIP LOCKED to prevent duplicate processing across multiple relay instances.
- NATS JetStream with per-subject delivery guarantees. Subject format: {module}.{entity}.{action} e.g. market.order.placed
- Consumers: all services subscribe to events they need. Consumers use durable subscriptions for reliable delivery after restarts.

- Dead Letter Queue: after 10 failed delivery attempts, event moves to DLQ subject and pages on-call engineer.
- Schema versioning: all event payloads include schema_version field. Consumers implement forward-compatible parsers.

5. Decision

✅ RFC-001 — Accepted (2024-07-15)

- All new module development must use the Transactional Outbox pattern for cross-module communication.
- Direct synchronous cross-service DB access is explicitly prohibited.
- Internal (within-module) synchronous calls are permitted and expected.

RFC-002 · EXCHANGE

Financial Transaction Safety Model

RFC-002

Financial Transaction Safety Model — Ledger & Escrow Design

Status: Accepted **Author:** Exchange Platform Team **Created:** 2024-06-15 **Updated:** 2024-08-01

Summary

This RFC defines the data model, invariants, and operational constraints for all financial transactions on the Kogi platform, including wallet management, the double-entry ledger, escrow lifecycle, and distribution mechanics. Financial correctness is non-negotiable; this RFC establishes the architectural constraints that enforce it.

1. Motivation

The Kogi Exchange System handles real money. Errors in financial logic — double-charges, incorrect balances, lost funds, or race conditions in concurrent balance updates — expose the company to legal liability, user harm, and terminal reputational damage. Standard application-level logic is insufficient; correctness must be enforced at the database, service, and infrastructure layers simultaneously.

2. Core Invariants

⚠️ Financial Invariants — These Must Never Be Violated

- INV-1: The sum of all ledger entries for a wallet must equal the wallet's current balance. This is verified by a nightly reconciliation job.
- INV-2: Ledger entries are IMMUTABLE. No UPDATE or DELETE is permitted on the ledger_entry table. Enforced by database trigger.
- INV-3: All wallet balance changes use SELECT FOR UPDATE to prevent race conditions. No balance is ever read-then-written outside a database transaction.
- INV-4: Escrow funds are never commingled with operational funds. Escrowed funds are tracked as fiat_reserved, not fiat_available.
- INV-5: Every monetary event emits a corresponding outbox event. No money moves without a traceable event record.
- INV-6: Idempotency keys are REQUIRED for all financial mutations. Duplicate requests return the cached original response, never execute twice.
- INV-7: All financial endpoints have stricter rate limits (30 req/min) than standard endpoints to reduce attack surface.

3. Double-Entry Ledger Design

Kogi uses a simplified single-party ledger (not full double-entry accounting), but with compensating entries for corrections. Every debit or credit creates a new row — never modifies an existing one. This produces a complete, auditable history of every balance change.

3.1 Wallet Balance Decomposition

Balance Field	Definition	Can Spend?	Modified By
fiat_available	Funds the user can spend or withdraw. Net of all settled transactions.	Yes	Deposits, escrow releases, withdrawals, refunds
fiat_reserved	Funds held in active outbound escrows. Cannot be spent until escrow resolves.	No	Escrow funding (+), escrow release/refund (-)
fiat_pending	Inbound funds not yet settled (e.g. bank transfer in transit).	No	Payment provider webhook on settlement
credit_balance	Platform service credits. Integer units. Not redeemable for cash.	Yes (in Market only)	Credit awards, credit spends

3.2 Concurrent Balance Safety

The following pattern is REQUIRED for all wallet mutations. Deviating from this pattern is a blocking code review issue.

Step	Code Pattern	Why
1. Lock wallet row	SELECT * FROM wallet WHERE id=$1 FOR UPDATE	Prevents any other transaction from reading or writing this wallet until this tx commits. Eliminates all race conditions.
2. Check balance	Application-level: if wallet.fiat_available < amount { return ErrInsufficientBalance }	DB-level check constraints are a fallback; application must check first for good error messages.
3. Modify balance	UPDATE wallet SET fiat_available=fiat_available-$2, version=version+1 WHERE id=$1 AND version=$3	version field provides optimistic lock as final safety net against stale reads.
4. Insert ledger entry	INSERT INTO ledger_entry (wallet_id, entry_type, amount, balance_after, ...) VALUES (...)	Immutable audit record written in same transaction as balance change.

Step	Code Pattern	Why
5. Commit	COMMIT	All of steps 1-4 are in one transaction. They all succeed or all fail.

4. Escrow State Machine

The escrow lifecycle is the most complex financial flow in the platform. The state machine is defined here as the authoritative reference.

Transition	Guard Conditions	Financial Effect	Event Emitted
∅ → CREATED	reference_id exists; amount > 0; buyer wallet KYC status checked	No funds moved yet; escrow record created	exchange.escrow.created
CREATED → FUNDED	Idempotency key present; buyer fiat_available ≥ amount	fiat_available -= amount; fiat_reserved += amount; ledger: HOLD entry	exchange.escrow.funded
FUNDED → DELIVERY_SUBMITTED	actor must be seller; delivery_notes NOT NULL	No funds moved; review_deadline set to now + review_window	exchange.escrow.delivery_submitted
DELIVERY_SUBMITTED → COMPLETE	buyer approved OR (now ≥ review_deadline AND no dispute)	fiat_reserved -= amount; seller fiat_available += (amount - fee); fee deducted; ledger: HOLD_RELEASE + CREDIT entries	exchange.escrow.released
DELIVERY_SUBMITTED → DISPUTED	dispute opened within review_deadline; reason + evidence required	No funds moved; auto-release timer frozen	exchange.escrow.disputed
DISPUTED → COMPLETE (seller)	Dispute resolved in seller's favor by mediator	Same as DELIVERY_SUBMITTED → COMPLETE	exchange.escrow.released
DISPUTED → REFUNDED (buyer)	Dispute resolved in buyer's favor; or seller unresponsive > 7d	fiat_reserved -= amount; buyer fiat_available += amount; ledger: HOLD_RELEASE + REFUND entries	exchange.escrow.refunded
FUNDED → REFUNDED	Order cancelled before delivery; governance approval if AO context	fiat_reserved -= amount; buyer fiat_available += amount	exchange.escrow.refunded

5. Alternatives Consi

dered for Concurrency Control

Option	Description	Pros	Cons	Verdict
A — SELECT FOR UPDATE(Proposed)	DB-level row lock on wallet during balance mutation	Absolute correctness; no lost updates possible; simple to reason about	Serializes writes to same wallet; potential throughput bottleneck for high-volume users	✅ Accepted — correctness > throughput for financial data
B — Optimistic Locking Only	Read wallet, check version, write with version check; retry on conflict	Higher throughput; no locks	Retry logic adds complexity; under high contention, retry storms; version check alone is insufficient for financial safety	❌ Rejected — too risky for financial data
C — Event Sourcing	Wallet balance derived from sum of all ledger entries, no mutable balance field	Immutable by construction; perfect auditability	Query complexity (every balance read requires SUM of all entries); performance at scale requires materialization anyway	⚠️ Partially adopted — ledger_entry is the source of truth; balance field is a materialized cache with nightly reconciliation

6. Decision

✅ RFC-002 — Accepted (2024-08-01)

- SELECT FOR UPDATE is mandatory for all wallet mutations. No exceptions. Enforced in code review checklist.
- ledger_entry is immutable at DB layer (trigger). Any workaround attempt is a security incident.
- Idempotency keys are required on all Exchange write endpoints. Missing Idempotency-Key returns 400, not 200.
- Nightly reconciliation job alerts on any wallet where balance ≠ sum(ledger entries).

RFC-003 · AI PLATFORM

AI Agent Context & Memory Architecture

RFC-003

AI Agent Context & Memory Architecture

Status: Draft **Author:** AI Platform Team **Created:** 2024-09-01 **Updated:** 2024-11-15

Summary

This RFC proposes the architecture for how the Kogi AI Agent manages user context across sessions, constructs prompts from portfolio data, and persists memory — balancing AI quality (more context = better results) against privacy (user data used in AI calls), cost (token count), and latency.

1. Motivation

The Kogi AI Agent's key differentiator is that it knows the user's full professional context — their portfolio, active projects, blocked stories, OKR status, and financial position. A general-purpose LLM given a bare user question has no access to this context. Kogi's value proposition requires making this context available to the LLM effectively and efficiently.

This RFC addresses three related but distinct questions: (1) What context is included in each LLM call? (2) How is conversation history managed across sessions? (3) Should the agent have persistent memory that carries across sessions, and if so, how?

2. Context Architecture Proposal

2.1 Per-Request Context Injection

For every AI Agent request, the system constructs a System Prompt that includes the user's current portfolio context. This is the most important mechanism for making the AI contextually aware.

Context Component	Data Source	Max Tokens	Included When
Portfolio summary	Portfolio service — item counts, health score, domain breakdown	200	Always
Active projects (top 5 by recency)	Portfolio service — name, status, % complete, due dates	400	Always
Blocked stories	WBS service — title, blocked reason, duration blocked	300	When user has blocked stories

Context Component	Data Source	Max Tokens	Included When
OKR status	Work service — active OKRs with current scores	300	When user has active OKRs
Recent AI actions (last 5)	Agent action log — type, result, timestamp	200	Always (prevents repeated actions)
Wallet summary	Exchange service — available balance, pending, active escrows	100	When user has Exchange enabled
Active sprint	WBS service — sprint goal, committed stories, days remaining	300	When user has active sprint
Custom user preferences	Profile — communication style, AI verbosity preference	100	Always

2.2 Conversation History

Conversation history within a session is included in the LLM call as the messages array. To manage token costs and context window limits, the system uses a sliding window with smart summarization.

Approach	Implementation	Token Budget	Trade-off
Last N turns (simple)	Include last 20 turns verbatim	~3,000 tokens	Simple; loses older context; recent context is high quality
Sliding window + summary (proposed)	Include last 10 turns verbatim + AI-generated summary of turns 11–50	~2,000 tokens	Best balance of recency + historical context
Hierarchical memory	Periodic consolidation: recent turns → working memory; working memory → long-term summary	~1,500 tokens	Best context compression; adds complexity; risk of information loss in summarization

2.3 Persistent Memory (Across Sessions)

Persistent memory — facts the agent remembers between separate sessions — is the most complex and most privacy-sensitive component of this proposal. The following three options are under active discussion.

Option	Description	Pros	Cons	Verdict
A — No persistentmemory	Each session starts fresh with only the injected portfolio context. No memory of past conversations.	Simple; clear privacy model; easy to explain to users	Lower quality assistance for users who have had many conversations; cannot learn user preferences over time	⚠️ Baseline — current default in draft
B — Opt-in semantic memory	Agent extracts "memory candidates" from each session (user preferences, stated goals, named people). Stored as embeddings. User can view and delete all memories.	High quality for opted-in users; transparent; user controls what is remembered	Implementation complexity (embedding storage, retrieval, pruning); privacy must be airtight	✅ Proposed — RFC author recommendation
C — Automatic persistent memory	Agent automatically maintains memory without explicit opt-in. Privacy notice in ToS.	Best AI quality; lowest friction	Privacy risk; user confusion; potential regulatory issue in GDPR jurisdictions (processing requires lawful basis); hard to reverse	❌ Not recommended

ℹ️ **Open Question — Requires Decision Before v1.0**

- Q1: Should AI memory be opt-in (B) or no persistent memory (A) for v1.0?
- Q2: If opt-in: what is the minimum memory capability that ships in v1.0 vs. v1.x?
- Q3: How do we handle GDPR right-to-erasure for embeddings stored in a vector database?
- Stakeholders: Product (user experience), Privacy/Legal (compliance), Engineering (implementation cost), AI team (quality impact)
- Decision needed by: 6 weeks before v1.0 code freeze

3. Cost & Latency Model

Configuration	Estimated Tokens/Request	Estimated Cost/1000 Requests	Latency Impact
No context injection	~1,000 tokens avg	$0.30 (claude-3-haiku)	Baseline
Portfolio context only (current proposal)	~2,500 tokens avg	$0.75 (claude-3-haiku)	+ 200ms token processing
Full context + conversation history	~4,000 tokens avg	$1.20 (claude-3-haiku)	+ 400ms token processing
Full context + memory retrieval	~5,000 tokens avg + embedding query	$1.50 + $0.01/embedding query	+ 600ms total

At $1.20/1,000 requests and Pro tier usage of 2,000 AI credits/month (assume 1 credit = 1 request), the AI cost per Pro user is $2.40/month against a $35/month subscription — a 6.9% COGS ratio. Acceptable.

4. Decision

⚠️ RFC-003 — Draft (Pending Decision on Memory Option)

- Per-request portfolio context injection: APPROVED for implementation (no open questions).
- Sliding window + summary for conversation history: APPROVED for implementation.
- Persistent memory architecture: OPEN — awaiting Product and Legal decision on Option A vs. B.
- Cost model reviewed and accepted by Finance. $35/mo Pro tier can absorb projected AI COGS.

RFC-004 · PRODUCT

Marketplace Seeding & Liquidity Bootstrap Strategy

RFC-004

Marketplace Seeding & Liquidity Bootstrap Strategy

Status: Accepted **Author:** Product + Market Team **Created:** 2024-08-01 **Updated:** 2024-09-15

Summary

Marketplaces require simultaneous supply and demand to provide value — the classic chicken-and-egg problem. This RFC proposes the specific strategy for seeding the Kogi marketplace at launch to avoid a cold-start failure, including supply acquisition, demand generation, category focus, and geographic sequencing.

1. The Liquidity Problem

A marketplace with buyers but no sellers offers no value. A marketplace with sellers but no buyers means providers leave. Both sides must reach a critical mass in the same categories at the same time. Generic marketplaces (all categories, all geographies, simultaneously) almost always fail at launch due to insufficient density in any one category.

2. Proposal — Category-First, Supply-Led Launch

Launch Phase	Focus	Supply Target	Demand Strategy	Success Metric
Pre-Launch(8 weeks before)	3 seeded categories: Technical Writing, Product Design, Full-Stack Development	50 hand-selected, verified providers per category (150 total). Criteria: 4.5+ rating on existing platform, responsive, English-speaking)	No public demand yet — invited buyers only (100 design agencies and tech consultancies)	150 active provider listings; 100 invited buyer accounts; at least 30 test transactions completed
Launch Day (Week 0)	Expand to 10 categories	300 providers across 10 categories. All	Public launch. PR + Product Hunt + HN. No	500 new signups in

		categories: minimum 20 verified providers before category goes live	ads at launch — earned media only.	first 7 days; $50K GMV in first 30 days; 0 "no match found" results for seeded categories
Month 1-3 (Demand-Led Expansion)	Unlock new categories based on buyer demand signals	Categories unlock when: ≥5 buyer requests for a category with no existing provider AND 3+ willing providers identified	Demand-signal page: buyers can "request" a category not yet available. Category unlocks when supply threshold is met.	$500K GMV by month 3; provider-to-buyer ratio of 1:3 or better in each active category
Month 3+ (Open Platform)	All categories available; self-serve provider onboarding	Any Kogi Pro user can create a listing. Verification remains required for "Verified Provider" badge.	Community-driven: top Kogi community members are highest-priority provider invites	$2M monthly GMV; 70%+ of GMV from organic (non-seeded) providers

3. Provider Onboarding Quality Gate

The marketplace's reputation depends on early provider quality. The following quality gates apply to the pre-launch and Launch Day phases only. Gates are removed for self-serve onboarding in Month 3+.

- Identity verified (government ID or LinkedIn account >2 years old with 50+ connections)
- Portfolio: at least 3 publicly viewable work samples with descriptions
- Response rate commitment: provider acknowledges 24-hour response time SLA during probationary period
- Minimum Kogi profile completeness: 90% (all required fields + bio + skills + avatar)
- Background check or existing platform verification: must have ≥10 reviews with ≥4.3 average on Upwork, Toptal, or similar, OR a professional reference check

4. Alternatives Considered

Option	Description	Pros	Cons	Verdict
A — Geographic focus(NYC only)	Launch only in New York City; seed dense local marketplace before expanding	Higher density in one place; easier to seed with events	Limits TAM at launch; independent work is already remote-first; geography is the wrong axis	❌ Rejected — remote work makes geography less relevant
B — Category	Launch with 3-10 categories fully seeded before expanding	Achievable supply goals; avoids zero-match	Slower to reach full marketplace; some users	✅ Accepted

focus(Proposed)		results; quality over quantity	will not find their category at launch	
C — Open marketplace immediately	All categories open from Day 1 with self-serve provider onboarding	Maximum supply speed	Quality race-to-bottom; buyers get zero-match results; first bad experience kills trust	❌ Rejected — quality is non-negotiable at launch
D — Kogi employees as seed supply	Hire 10-20 contractors to fill seed supply roles in all categories	Full control of quality and availability	Expensive; doesn't create real marketplace network effects; unsustainable	❌ Rejected

5. Decision

✅ RFC-004 — Accepted (2024-09-15)

- Category-first, supply-led launch strategy adopted.
- Pre-launch supply target: 150 verified providers across 3 seeded categories.
- Quality gates for pre-launch providers are mandatory. No exceptions.
- Category demand-signal page ships with v1.0 to gather expansion signals from Day 1.

RFC-005 · GOVERNANCE

Cooperative & AO Governance Engine Design

RFC-005

Cooperative & AO Governance Engine Design

Status: Draft **Author:** AO Platform Team **Created:** 2024-10-01 **Updated:** 2024-12-01

Summary

This RFC proposes the design of the Autonomous Organization (AO) governance engine — the system that manages proposals, voting, quorum calculation, outcome execution, and governance audit logs for worker cooperatives, investment clubs, and other collective organizations on the Kogi platform.

1. Motivation

Cooperative and AO governance is one of Kogi's three primary differentiators (alongside integrated payments and contextual AI). No existing platform provides software infrastructure specifically designed for worker cooperatives and member-owned organizations. Getting this right — legally sound, technically robust, and genuinely easy to use — is the basis of a significant competitive moat.

2. Governance Models Supported

Governance Model	Voting Weight	Use Case	Quorum Calculation
One-Member-One-Vote (1M1V)	Each member has exactly 1 vote regardless of stake, contribution, or tenure	Worker cooperatives (most common); hobbyist collectives	Quorum = (votes cast / eligible_member_count) ≥ quorum_pct
Stake-Weighted	Voting weight proportional to equity stake or capital contribution	Investment clubs; capital cooperatives	Quorum = (weighted votes cast / total eligible voting weight) ≥ quorum_pct
Hybrid	Configurable mix: e.g. 50% stake-weighted + 50% 1M1V, blended at vote tally	Complex cooperatives with both labor and capital members	Two quorum calculations run in parallel; both must be met
Role-Based	Different member roles have different voting rights (e.g. Full Member votes on all; Associate	Nonprofits; tiered membership orgs	Quorum calculated separately per eligible_role_set for each proposal type

	votes on operational but not charter)		

3. Proposal Lifecycle & Execution

The most important design decision in this RFC is: when a proposal passes, how are the downstream effects executed? Options range from fully manual (humans execute) to fully automated (system executes without further human interaction).

Option	Description	Pros	Cons	Verdict
A — Manual execution	Proposal passes → notification sent → designated executor performs action manually	Maximum flexibility; no automation complexity; easy to audit	Human delays; human errors; no enforcement of outcomes; defeats the value of formal governance	❌ Rejected — governance without enforcement is just a chat thread
B — Semi-automated(Proposed)	Proposal passes → outcome_type determines action → system executes reversible actions automatically; irreversible actions require one additional human confirmation	Good balance: automation for routine outcomes (capital transfer, member admission), human confirmation for extreme outcomes (dissolution, mass removal)	More complex state machine; need to classify outcomes by reversibility	✅ Proposed
C — Fully automated	All outcomes execute automatically with no additional confirmation after vote passes	Maximum efficiency; strong enforcement	Higher risk of unintended consequences; irreversible outcomes (dissolving an org, removing all members) could be catastrophic if there was a governance attack or error	⚠️ Partially accepted — adopted for low-risk outcomes only (capital transfers < threshold, routine member admission)

3.1 Outcome Execution Classification

Proposal Type	Execution Class	Auto-Execute?	Human Confirmation Required?
Capital allocation (< $1,000)	Low risk, reversible via new proposal	Yes — within 1 hour of vote close	No
Capital allocation (≥ $1,000)	Medium risk, requires	Yes — within 1 hour	No, but alert all admins with 2-hour override window

Proposal Type	Execution Class	Auto-Execute?	Human Confirmation Required?
	significant fund movement		
Member admission	Low risk, reversible via removal proposal	Yes — provisioning triggered automatically	No (member receives invitation)
Member removal	High risk, partially irreversible (access revoked)	Partial — access suspended automatically; fund return calculated	Yes — designated admin must confirm suspension converts to permanent removal
Charter amendment	High risk, resets governance rules	No — system updates charter on confirmation only	Yes — AO owner must re-confirm after vote to prevent accidental amendment
AO dissolution	Extreme risk, irreversible	No — never auto-executed	Yes — 72-hour cooling off period + all admins must confirm

4. Governance Audit Log — Immutability

The governance audit log is the most legally sensitive component of the governance engine. For cooperatives, accurate records of proposals, votes, and outcomes may be required by applicable cooperative law. The audit log must be:

- Append-only: no UPDATE or DELETE on governance records. Enforced at the DB trigger layer (same pattern as financial ledger in RFC-002).
- Timestamped with high precision: TIMESTAMPTZ nanosecond precision; server-side timestamp cannot be client-supplied.
- Exportable: members can export full governance history in JSON and CSV at any time.
- Cryptographically linked (future): a future enhancement (RFC-005-b) will anchor monthly governance log hashes to a public blockchain for legal verifiability. This is NOT in scope for v1.0.

5. Open Questions (Blocking for v1.0)

ℹ️ Open Questions — RFC-005

- Q1: What legal review is required before shipping AO governance for real cooperatives? Engaging cooperative law specialists is required before launch of this module for production use.
- Q2: Should charter templates be written by Kogi or by third-party cooperative law organizations (e.g. USFWC)? Kogi-authored templates carry legal risk; partnership with USFWC is strongly recommended.
- Q3: What is the minimum viable governance feature set for v1.0? Current proposal: 1M1V only, proposals + voting, 2 proposal types (capital allocation + member admission/removal). Charter builder and stake-weighted models deferred to v1.x.
- Q4: How do we handle the governance of the governance system itself? (Who can change AO charter parameters? Current proposal: charter amendment proposal type, requiring supermajority.)

6. Decision

⚠️ RFC-005 — Draft (Pending Legal Review)

- Semi-automated execution (Option B) adopted for proposal outcome execution.
- Execution classification table (Section 3.1) is the authoritative reference for auto-execute decisions.
- Governance audit log immutability pattern from RFC-002 is adopted verbatim.
- v1.0 scope: 1M1V voting only, 2 proposal types, no charter builder. All other models in v1.x.
- Legal review of cooperative charter templates required before module ships to production.

RFC-006 · ENGINEERING

Monorepo, Build System & Service Extraction Strategy

RFC-006

Monorepo, Build System, and Service Extraction Strategy

Status: Accepted **Author:** Platform Architecture Team **Created:** 2024-07-01 **Updated:** 2024-08-15

Summary

This RFC establishes the monorepo structure, build system (Bazel), module boundary conventions, and the strategy for extracting independent services from the initial monorepo over time. The goal is to start with a well-structured monolith and evolve to microservices without a painful rewrite.

1. Proposal — Bazel Monorepo with Explicit Module Boundaries

All Kogi code — Zig kernel, Go services, Scala analytics, TypeScript web, Java desktop — lives in a single monorepo managed by Bazel. This provides: atomic commits across services, shared tooling, single CI pipeline, and consistent versioning. Critically, explicit build targets enforce module boundaries: services cannot import each other's internal packages directly.

1.1 Directory Structure

Directory	Language	Purpose
kernel/	Zig	Platform kernel — module registry, event bus adapter, scheduler, FFI interface
services/api/	Go	API gateway — routing, auth middleware, rate limiting
services/portfolio/	Go	Portfolio service — domain logic, gRPC server, event handlers
services/exchange/	Go	Exchange service — wallet, escrow, ledger, distributions
services/market/	Go	Market service — listings, orders, search integration

Directory	Language	Purpose
services/community/	Go	Community service — spaces, rooms, WebSocket hub, social graph
services/wbs/	Go	WBS service — stories, sprints, WBS hierarchy
services/studio/	Go	Idea Studio service — ideas, pipeline, AI synthesis
services/work/	Go	Work & OKR service — OKRs, automation rules, capacity
services/fund/	Go	Crowdfunding service — campaigns, capital pools
services/ao/	Go	AO governance service — proposals, voting, charter
services/ai/	Go	AI agent service — LLM integration, context, action log
analytics/	Scala	Analytics engine — metrics, health scoring, reporting
web/	TypeScript/Angular	Single-page web application
desktop/	Java/JavaFX	Native desktop application
proto/	Protobuf	Shared gRPC service definitions and event schemas
shared/go/	Go	Shared libraries — logging, errors, pagination, auth middleware
shared/proto/	Protobuf	Shared message types — Money, UUID, Timestamp
infra/	Terraform/Helm	Infrastructure-as-code — AWS, Kubernetes, NATS, Elasticsearch
scripts/	Bash/Python	Developer tooling, CI scripts, migration utilities

1.2 Module Boundary Rules

- A service may import from shared/go/ (shared libraries)
- A service may import generated protobuf code from proto/ (for gRPC clients)
- A service may NOT import another service's internal packages directly — this is enforced by Bazel visibility rules
- Cross-module communication is ONLY via: (a) gRPC calls using proto/ definitions, or (b) NATS events using outbox pattern (RFC-001)

- Database schemas are service-private. No cross-service joins. No shared DB schemas.

2. Service Extraction Criteria

Today, all services run in the same Kubernetes deployment for operational simplicity. A service should be extracted to independent deployment when it meets any of the following criteria:

Criterion	Example	Action
Independent scaling requirement	Community service needs 10× more instances than Portfolio service during peak hours	Extract to independent Kubernetes Deployment; give independent HPA rules
Independent tech stack requirement	Analytics service uses Scala/ClickHouse; cannot share Go service deployment	Already extracted (analytics/ is a separate Deployment from day one)
Independent release cadence	Exchange service requires weekly audit-gated releases; other services release daily	Extract to independent CI/CD pipeline with additional gates
Independent failure isolation requirement	Exchange service down must not affect Community (chat) availability	Extract with dedicated DB, dedicated NATS consumer group
Regulatory isolation requirement	Financial data must be in a separate data residency region	Extract Exchange to EU-specific deployment independent of global services

3. Decision

✅ RFC-006 — Accepted (2024-08-15)

- Bazel monorepo is the authoritative build system. No individual service repos.
- All boundary rules (Section 1.2) are enforced via Bazel visibility and CI lint checks.
- Service extraction follows criteria in Section 2; no premature extraction.
- Exchange service is extracted from day one due to financial isolation requirements.

RFC-007 · PRODUCT

Freemium Tier Definition & Feature Gating Architecture

RFC-007

Freemium Tier Definition & Feature Gating Architecture

Status: Accepted **Author:** Product + Engineering **Created:** 2024-09-01 **Updated:** 2024-10-15

Summary

This RFC defines the exact feature boundaries between pricing tiers and specifies the technical architecture for feature gating — how the platform enforces plan limits at the API and UI layers without requiring code changes to add or modify tier limits.

1. Motivation

Poorly designed freemium tiers kill products in two ways: too generous, and users never convert to paid; too restrictive, and users never experience enough value to want to pay. This RFC establishes the philosophy and specific boundaries. It also specifies the technical mechanism so that tier boundaries can be changed via configuration without code deploys.

2. Feature Gating Philosophy

- Gate on value creation, not on access. Users should be able to create value on the free tier; they should hit limits only when they want more of that value.
- Never gate critical functionality that would leave a user's data inaccessible. A user who has 20 portfolio items should never be unable to view them if they downgrade.
- Gate on quantity, not quality. Free users get fewer AI credits, not worse AI responses. Free users get fewer portfolio items, not a crippled portfolio viewer.
- The upgrade prompt should happen at the moment the user most wants to do more — not as a nagging banner.

3. Feature Gate Definitions

Feature	Free	Starter ($15)	Pro ($35)	Team ($80)	Coop ($200)
Portfolio items (total)	10	50	Unlimited	Unlimited	Unlimited
Active projects	3	10	Unlimited	Unlimited	Unlimited

WBS stories per project	25	200	Unlimited	Unlimited	Unlimited
Idea Studio stages	1–2 only	1–3	All 5 stages	All 5 stages	All 5 stages
AI Agent credits/month	20	200	2,000	2,000/member	5,000/org
Automation rules	0	5	Unlimited	Unlimited	Unlimited
Market — buy listings	Yes	Yes	Yes	Yes	Yes
Market — sell listings	0	0	Yes	Yes	Yes
Exchange — receive payments	0	Yes (Stripe link only)	Full Exchange	Full Exchange	Full + distributions
Exchange — escrow	No	Yes	Yes	Yes	Yes
Community — join/create Spaces	Unlimited	Unlimited	Unlimited	Unlimited	Unlimited
Community — DM	10/day	Unlimited	Unlimited	Unlimited	Unlimited
OKRs	3 personal	10 personal	Unlimited personal	Unlimited + team OKRs	Unlimited + org OKRs
AO Governance	No	No	View only	Create + vote	Full admin
Crowdfunding campaigns	0	1 reward/donation	3 campaigns	5 campaigns	Unlimited
Analytics	Basic (last 30d)	Standard (last 90d)	Full (all time)	Full + team	Full + org
File storage (attachments)	500MB	5GB	50GB	100GB	500GB
API access	No	Read-only	Full	Full	Full + webhooks

4. Technical Feature Gating Architecture

Feature gates are evaluated at the API layer — the JWT contains the user's plan, and every service reads plan limits from a central FeatureGate service rather than hard-coding limits in application code. This means tier limits can be changed via a single config update without service deploys.

4.1 Architecture

Component	Responsibility	Implementation
Plan field in JWT	Carries current plan for each request; cached in token for 24h	plan: "pro" | "starter" | "free" | "team" | "coop" | "enterprise"

Component	Responsibility	Implementation
FeatureGate service	Central repository of plan limits; serves limit lookups via gRPC	YAML-based config per plan; reloaded without service restart
Gate middleware (per service)	Intercepts write operations; checks current usage against plan limits; returns 402 PLAN_LIMIT_REACHED on violation	Reads plan from JWT + current usage from service DB; calls FeatureGate for limit
Usage tracking	Each service tracks current usage of gated resources in a usage_counters table	Incremented on create; decremented on delete; never goes negative
Upgrade prompt event	When a gate is hit, a plan_limit_hit event is emitted with the resource type	Used by frontend to show the appropriate upgrade modal with specific value messaging

5. Downgrade Behavior

Scenario	Behavior
User downgrades; they have 30 portfolio items but free tier allows 10	Existing items are NOT deleted. User can view all 30 items but cannot create new items until they are below the limit. Banner shown: "You have 20 items above your plan limit. Upgrade or archive items to continue creating."
User downgrades; they have active automation rules but free tier allows 0	Rules are paused, not deleted. User sees paused rules with upgrade prompt to re-activate.
User downgrades from Pro; they have active Market seller listings	Listings are set to paused status. User cannot accept new orders but existing orders are fulfilled.
User's payment fails; plan downgrades to grace period	7-day grace period: all Pro features available, banner shown. After 7 days: rules above Free limits are paused; user can still access all data.

6. Decision

✅ RFC-007 — Accepted (2024-10-15)

- Feature gate table (Section 3) is the product specification for all tier limits. Engineering implements from this table.
- FeatureGate service is a first-class service; no hard-coded plan limits in business logic.
- Downgrade behavior (Section 5) is the UX standard; no data deletion on downgrade.
- Gate middleware logs plan_limit_hit events to analytics for upgrade funnel analysis.

RFC-008 · REAL-TIME

Real-Time Collaboration Architecture

RFC-008

Real-Time Collaboration Architecture — WebSocket vs. SSE vs. Polling

Status: Accepted **Author:** Community Platform Team **Created:** 2024-08-15 **Updated:** 2024-09-30

Summary

This RFC evaluates three approaches for delivering real-time updates to clients (WebSocket, Server-Sent Events, and long-polling) and recommends a hybrid approach: WebSocket for high-frequency bidirectional communication (chat rooms) and SSE for lower-frequency unidirectional updates (notifications, AI agent responses, story status changes).

1. Requirements

Use Case	Direction	Update Frequency	Latency Target	Message Volume
Live chat room messages	Bidirectional	High (multiple per second per active room)	< 300ms	Very high — up to 10K msgs/sec at scale
Collaborative WBS board updates	Server → Client	Medium (multiple per minute per board)	< 1 second	High — 100K stories × concurrent editors
AI Agent streaming responses	Server → Client	High (tokens streaming for 5–30 seconds)	First token < 3 seconds	Medium — per-user, short duration
Notifications (new proposal, story blocked, escrow funded)	Server → Client	Low (few per hour)	< 5 seconds	Low — high fan-out per event, low frequency
Presence (who is online in a Space)	Bidirectional	Medium (on connect/disconnect)	< 2 seconds	Medium
Story status updates on Kanban board	Server → Client	Low-medium	< 2 seconds	Medium

2. Options Analysis

Option	Description	Pros	Cons	Verdict
A — WebSocket everywhere	Maintain persistent bidirectional WS connection for all real-time updates	Single connection handles all cases; bidirectional native; native binary support	High connection count at scale (1 WS connection per tab per user); complex state management on reconnect; overkill for low-frequency notifications	⚠️ Too heavy for all cases
B — SSE everywhere	Server-sent events for all real-time delivery; client sends updates via REST	Simpler than WebSocket; automatic reconnect built in; works with HTTP/2 multiplexing; browser-native	Unidirectional only — chat messages cannot use SSE natively (require separate REST POST + SSE receive); higher latency for chat	⚠️ Insufficient for chat
C — Polling	Client polls API on interval for updates	Simplest to implement; works everywhere; no connection management	Latency proportional to poll interval; wasteful bandwidth; poor for chat; 5-second polling is industry-standard bad UX for messaging	❌ Rejected for all real-time use cases
D — Hybrid: WS for chat + SSE for everything else (Proposed)	WebSocket: Community chat rooms and presence. SSE: AI streaming, notifications, board updates, story status.	Right tool for right job; WS overhead only where bidirectional is required; SSE simplicity for most use cases	Two connection mechanisms to maintain; more complex client state management	✅ Accepted

3. Hybrid Architecture Detail

3.1 WebSocket — Chat & Presence

- Connection: one WS connection per active room (not per tab). Connection multiplexed across rooms using room subscription messages.
- Multi-pod fan-out: NATS subject per room (room.{roomID}). All pods subscribed. Message from client on pod A → pod A publishes to NATS → all pods fan out to local WS clients in that room.
- Reconnection: client implements exponential backoff (100ms, 200ms, 400ms... max 10s). On reconnect, client sends last_seen_message_id and server replays missed messages.
- Presence: heartbeat every 30 seconds. Disconnect detected after 2 missed heartbeats (60 seconds). Presence state stored in Redis with 90-second TTL.

3.2 Server-Sent Events — Notifications & AI Streaming

- Connection: one SSE connection per authenticated user session. Receives all non-chat real-time events for that user.
- AI streaming: AI Agent response is streamed via SSE using delta events (text_delta, tool_call, complete). SSE is ideal here — unidirectional, long-lived during a response, browser handles reconnect.
- Notification fan-out: when an event (e.g. story.blocked) fires for user X, the notification service pushes a notification_ready SSE event to user X's open SSE connection.
- HTTP/2: SSE connections use HTTP/2 to avoid the 6-connection-per-domain browser limit that affects HTTP/1.1 SSE implementations.

3.3 Connection Lifecycle

Event	WebSocket Behavior	SSE Behavior
Initial connect	Authenticate via token query param; subscribe to room(s)	Authenticate via Bearer header; register user connection in Redis
Token expiry	Send auth_refresh event; client sends new token in message	SSE connection closes with 401; client re-authenticates and reconnects
Network interruption	Client reconnects with backoff; replays missed messages via last_seen_id	Browser auto-reconnects SSE natively; server replays queued notifications from last event_id
Server pod restart	NATS subscription ensures messages are not lost; reconnecting clients replay from NATS	Notification queue in Redis ensures no notifications lost during reconnect window
Client closes tab	WS closed; server detects disconnect; presence updated	SSE closed; server removes connection registration from Redis

4. Scale Model

Metric	Estimate (100K MAU)	Estimate (1M MAU)	Infrastructure Response
Peak concurrent WS connections	15,000 (15% DAU/MAU × peak hour factor)	150,000	Auto-scale community-svc; 2K connections/pod limit; 75 pods peak at 1M MAU
Peak concurrent SSE connections	40,000 (40% DAU online at peak)	400,000	SSE is stateless per HTTP/2 connection; 5K connections/pod; 80 pods peak at 1M MAU

NATS messages/sec (chat)	5,000	50,000	NATS cluster handles 10M+ msgs/sec; no bottleneck until 100M MAU
Redis presence keys	40,000 (one per SSE connection)	400,000	Redis Cluster easily handles 10M+ keys; no bottleneck

5. Decision

✅ RFC-008 — Accepted (2024-09-30)

- Hybrid architecture adopted: WebSocket for chat/presence, SSE for notifications/AI streaming/board updates.
- NATS-based fan-out for multi-pod WebSocket delivery (as specified in LLSDS Section 5.4).
- SSE uses HTTP/2; HTTP/1.1 SSE fallback for legacy clients only.
- Scale model reviewed; no bottlenecks projected before 1M MAU on current architecture.

INDEX · REFERENCE

RFC Cross-Reference Matrix

Decision Dependencies

RFC	Depends On	Depended On By
RFC-001 — Event-Driven Architecture	—	RFC-002, RFC-003, RFC-005, RFC-008 (all use event bus)
RFC-002 — Financial Safety Model	RFC-001 (outbox for financial events)	RFC-004 (marketplace escrow), RFC-005 (capital pool), RFC-007 (plan downgrade behavior)
RFC-003 — AI Context Architecture	RFC-001 (AI events), RFC-006 (monorepo structure)	RFC-007 (AI credits as a gated feature)
RFC-004 — Marketplace Seeding	RFC-002 (escrow required for marketplace)	—
RFC-005 — AO Governance	RFC-001 (governance events), RFC-002 (capital pool financial model)	—
RFC-006 — Monorepo & Build	—	RFC-003 (service extraction), all engineering work
RFC-007 — Feature Gating	RFC-006 (FeatureGate as a service in monorepo)	RFC-003 (AI credits), RFC-004 (marketplace access)
RFC-008 — Real-Time Architecture	RFC-001 (NATS for WS fan-out), RFC-006 (community-svc in monorepo)	RFC-005 (governance vote notifications)

Open Items Summary

RFC	Open Item	Owner	Deadline
RFC-003	Decision: opt-in AI memory (Option B) vs. no persistent memory (Option A) for v1.0	Product + Privacy/Legal	6 weeks before v1.0 code freeze
RFC-005	Legal review of AO governance module before production launch to real cooperatives	Legal + AO Team	3 months before v1.0 launch

RFC	Open Item	Owner	Deadline
RFC-005	Minimum viable governance scope for v1.0: confirm 1M1V + 2 proposal types is sufficient	Product	1 month before feature code freeze
RFC-003	GDPR lawful basis analysis for AI persistent memory feature	Privacy/Legal	With RFC-003 memory decision

Kogi Platform — Request for Comments Collection

RFC-001 through RFC-008 · v1.0 · Confidential — Engineering & Product